

**Міністерство освіти і науки України  
Національний технічний університет України  
«Київський політехнічний інститут імені Ігоря Сікорського»  
Факультет інформатики та обчислювальної техніки  
Кафедра обчислювальної техніки**

**Лабораторна робота №1**

з дисципліни  
«Алгоритми і структури даних»

Виконала:

студентка групи ІМ-44  
Крошка Дар'я Олександрівна  
номер у списку групи: 11

Перевірів:

Молчанов А. С.

Київ 2025

### **Завдання:**

Створити список з  $n$  ( $n > 0$ ) елементів ( $n$  вводиться з клавіатури), якщо інша кількість елементів не вказана у конкретному завданні за варіантом. Тип ключів (інформаційних полів) задано за варіантом. Вид списку (черга, стек, дек, прямий однозв'язний лінійний список, обернений однозв'язний лінійний список, двозв'язний лінійний список, однозв'язний кільцевий список, двозв'язний кільцевий список) вибрати самостійно з метою найбільш доцільного розв'язку поставленої за варіантом задачі. Створити функції (або процедури) для роботи зі списком (для створення, обробки, додавання чи видалення елементів, виводу даних зі списку в консоль, звільнення пам'яті тощо). Значення елементів списку взяти самостійно такими, щоб можна було продемонструвати коректність роботи алгоритму програми. Введення значень елементів списку можна виконати довільним способом (випадкові числа, формування значень за формулою, введення з файлу чи з клавіатури). Виконати над створеним списком дії, вказані за варіантом, та коректне звільнення пам'яті списку. При виконанні заданих дій, виводі значень елементів та звільненні пам'яті списку вважати, що довжина списку (кількість елементів) невідома на момент виконання цих дій. Тобто, не дозволяється зберігати довжину списку як константу, змінну чи додаткове поле.

### **Варіант:**

Ключами елементів списку є латинські літери. Відсортувати елементи списку у лексикографічному порядку, не використовуючи додаткових структур даних, крім простих змінних (тобто «на тому ж місці»), методом вставки.

### **Текст програми:**

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
```

```
typedef struct Node {  
    char key;  
    struct Node *next;  
} Node;
```

```
Node* createNode(char key) {  
    Node* newNode = (Node*)malloc(sizeof(Node));  
    newNode->key = key;  
    newNode->next = NULL;  
    return newNode;  
}
```

```
void append(Node** head, char key) {  
    Node* newNode = createNode(key);  
    if (*head == NULL) {  
        *head = newNode;  
        return;  
    }  
    Node* temp = *head;  
    while (temp->next) {  
        temp = temp->next;  
    }  
    temp->next = newNode;  
}
```

```
void printList(Node* head) {  
    while (head) {  
        printf("%c ", head->key);  
        head = head->next;  
    }  
    printf("\n");  
}
```

```
}
```

```
void insertionSort(Node** head) {  
    if (*head == NULL || (*head)->next == NULL) return;  
  
    Node* sorted = NULL;  
    Node* current = *head;  
  
    while (current) {  
        Node* next = current->next;  
        if (sorted == NULL || sorted->key >= current->key) {  
            current->next = sorted;  
            sorted = current;  
        } else {  
            Node* temp = sorted;  
            while (temp->next && temp->next->key < current->key) {  
                temp = temp->next;  
            }  
            current->next = temp->next;  
            temp->next = current;  
        }  
        current = next;  
    }  
    *head = sorted;  
}
```

```
void freeList(Node* head) {  
    while (head) {  
        Node* temp = head;  
        head = head->next;  
        free(temp);  
    }
```

```
}  
}
```

```
int main() {  
    int n;  
    printf("Enter the number of elements: ");  
    scanf("%d", &n);  
  
    Node* head = NULL;  
    printf("Enter %d letters: ", n);  
    for (int i = 0; i < n; i++) {  
        char key;  
        scanf(" %c", &key);  
        if (!isalpha(key)) {  
            printf("Invalid input. Only letters allowed.\n");  
            i--;  
            continue;  
        }  
        append(&head, key);  
    }  
  
    printf("Original list: ");  
    printList(head);  
  
    insertionSort(&head);  
  
    printf("Sorted list: ");  
    printList(head);  
  
    freeList(head);  
    return 0;  
}
```

}

## Тестування програми:

```
"C:\Users\Asus\Desktop\ASD2" x + v
Enter the number of elements: 26
Enter 26 letters: qwertyuiopasdfghjklzxcvbnm
Original list: q w e r t y u i o p a s d f g h j k l z x c v b n m
Sorted list: a b c d e f g h i j k l m n o p q r s t u v w x y z

Process returned 0 (0x0)    execution time : 28.698 s
Press any key to continue.
```

```
"C:\Users\Asus\Desktop\ASD2" x + v
Enter the number of elements: 5
Enter 5 letters: dbcsa
Original list: d b c s a
Sorted list: a b c d s

Process returned 0 (0x0)    execution time : 8.587 s
Press any key to continue.
```

```
"C:\Users\Asus\Desktop\ASD2" x + v
Enter the number of elements: 10
Enter 10 letters: zqxcvrbt
Original list: z q x w c e v r b t
Sorted list: b c e q r t v w x z

Process returned 0 (0x0)    execution time : 4.932 s
Press any key to continue.
```

```
"C:\Users\Asus\Desktop\ASD2" × + ∨  
Enter the number of elements: 7  
Enter 7 letters: AFCRLHY  
Original list: A F C R L H Y  
Sorted list: A C F H L R Y  
  
Process returned 0 (0x0)    execution time : 22.115 s  
Press any key to continue.
```

```
"C:\Users\Asus\Desktop\ASD2" × + ∨  
Enter the number of elements: 6  
Enter 6 letters: a,b,c,d,t,w  
Invalid input. Only letters allowed.  
Invalid input. Only letters allowed.  
Invalid input. Only letters allowed.  
Invalid input. Only letters allowed.  
Invalid input. Only letters allowed.  
Original list: a b c d t w  
Sorted list: a b c d t w  
  
Process returned 0 (0x0)    execution time : 14.185 s  
Press any key to continue.
```

```
"C:\Users\Asus\Desktop\ASD2" × + ∨  
Enter the number of elements: 4  
Enter 4 letters: edle  
Invalid input. Only letters allowed.  
  
ed, eg  
Original list: e d e e  
Sorted list: d e e e  
  
Process returned 0 (0x0)    execution time : 24.847 s  
Press any key to continue.
```

## **Висновки:**

У цій роботі було реалізовано однозв'язний лінійний список, що складається з **n** елементів, де ключами є латинські літери. Користувач вводить кількість елементів і самі значення з клавіатури, після чого виконується сортування методом вставки без використання додаткових структур даних.

Для зручності та структурованості було створено окремі функції для виконання основних операцій:

- Створення та додавання елементів у список.
- Вивід списку до та після сортування.
- Сортування методом вставки "на місці", без використання масивів або додаткових змінних для збереження довжини списку.

Реалізований алгоритм відповідає всім вимогам завдання. Він демонструє правильну роботу з однозв'язним списком, коректно виконує сортування та ефективно керує пам'яттю. При цьому його можна доповнити та оптимізувати, зробивши ще більш універсальним та продуктивним.